Teoria

Raccolta Completa: 2 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Ricerca Lineare**: controlla cella per cella. Funziona su **qualsiasi array** (anche disordinato). Complessità: **$O(n)$**.
> - **Ricerca Binaria (Dicotomica)**: dimezza lo spazio di ricerca (*Divide et Impera*). Richiede **array obbligatoriamente già ordinato**! Complessità: **$O(\log n)$**.
> - **Confronto Potenza**: su 1 milione di elementi la lineare fa fino a 1.000.000 di confronti; la binaria ne fa **al massimo 20**!

Cercare se un elemento è presente all'interno di una collezione di dati è un problema cardine dell'informatica.

In C++ gli algoritmi di ricerca standard su vettori sono due:

1. **Ricerca Lineare (o Sequenziale)**: elementare, applicabile a qualsiasi array.
2. **Ricerca Binaria (o Dicotomica)**: straordinariamente veloce, ma richiede che l'array sia già ordinato.

---

1. Ricerca Lineare (Sequenziale)

Come Funziona

Scorre il vettore cella per cella dall'indice `0` all'indice `N - 1`. Non appena trova il valore cercato restituisce l'indice corrente; se termina il vettore senza successo restituisce `-1`.

```
flowchart LR
    subgraph ARRAY ["Array disordinato: [14, 5, 27, 42, 8] - Chiave cercata: 42"]
        direction LR
        C0["[0]: 14<br>(No)"] --> C1["[1]: 5<br>(No)"] --> C2["[2]: 27<br>(No)"] -->
    end
```

```
int ricercaLineare(const int arr[], int dimensione, int chiave) {
    for (int i = 0; i < dimensione; i++) {
        if (arr[i] == chiave) {
            return i; // Trovato all'indice i
        }
    }
    return -1; // Non presente
}
```

Complessità Computazionale

- **Caso Migliore**: O(1) — L'elemento cercato è il primo esaminato.
- **Caso Peggiore e Medio**: $O(n)$ — L'elemento è l'ultimo o assente (dobbiamo esaminare tutti gli n elementi).

---

2. Ricerca Binaria (Dicotomica)

> ❓ **Domanda d'Esame: Qual è il prerequisito della ricerca binaria?**
>
> La ricerca binaria richiede **tassativamente che il vettore sia già ordinato** (in ordine crescente o decrescente). Se l'array è disordinato, l'algoritmo fallirà!

Logica del "Divide et Impera"

Invece di controllare un elemento alla volta, a ogni confronto **dimezziamo lo spazio di ricerca**:

```
flowchart TD
    Passo1["1. Calcola indice mediano: medio = inizio + (fine - inizio) / 2"] --> Co
    Confronto -->|"Sì"| Trovato["Elemento Trovato! Restituisci 'medio'"]
    Confronto -->|"No: chiave < arr[medio]"| Sinistra["Cerca a sinistra:<br>fine = m
    Confronto -->|"No: chiave > arr[medio]"| Destra["Cerca a destra:<br>inizio = med
    Sinistra --> Ripeti{"inizio <= fine?"}
    Destra --> Ripeti
    Ripeti -->|"Sì"| Passo1
    Ripeti -->|"No"| NonTrovato["Dato Assente: Restituisci -1"]
```

> 💡 **Formula Sicura contro l'Integer Overflow**
>
> Calcolando `(inizio + fine) / 2` su array enormi si rischia un **Integer Overflow**.
> Usa sempre la formula sicura: **`medio = inizio + (fine - inizio) / 2`**.

Codice C++ (Versione Iterativa)

```
int ricercaBinaria(const int arr[], int dimensione, int chiave) {
    int inizio = 0;
    int fine = dimensione - 1;

    while (inizio <= fine) {
        int medio = inizio + (fine - inizio) / 2;

        if (arr[medio] == chiave) {
            return medio; // Trovato!
        }

        if (arr[medio] < chiave) {
            inizio = medio + 1; // Sposta a destra
        } else {
            fine = medio - 1;   // Sposta a sinistra
        }
    }

    return -1; // Non presente
}
```

> 🖼️ **Placeholder Immagine: Schema visivo ad albero del dimezzamento per la ricerca binaria**
>
> *Suggerimento per Obsidian: inserisci qui un disegno che illustra il restringimento progressivo dei puntatori inizio, medio e fine.*
> `![[Pasted image binary_search_tree.png|550]]`

---

3. Confronto di Efficienza: Lineare vs Binaria

La complessità temporale della ricerca binaria è logaritmica: **O(log2 n)**.

Dimensione Vettore (n)	Passi Massimi (Lineare O(n))	Passi Massimi (Binaria O(log n))
10 elementi	10	4
1.000 elementi	1.000	10
1.000.000 (1 milione)	1.000.000	**solo 20 passi!**
1.000.000.000 (1 miliardo)	1.000.000.000	**solo 30 passi!**

🧪 Programma Completo di Test Eseguibile (Clicca per espandere)

```cpp
#include <iostream>
using namespace std;

int ricercaBinaria(const int arr[], int dim, int chiave);

int main() {
    const int N = 7;
    int dati[N] = {4, 9, 15, 23, 38, 51, 77}; // Array ordinato

    int bersaglio = 38;
    int pos = ricercaBinaria(dati, N, bersaglio);

    if (pos != -1) cout << "Trovato all'indice: " << pos << endl;
    else cout << "Non trovato!" << endl;
    return 0;
}

int ricercaBinaria(const int arr[], int dim, int chiave) {
    int inizio = 0, fine = dim - 1;
    while (inizio <= fine) {
        int medio = inizio + (fine - inizio) / 2;
        if (arr[medio] == chiave) return medio;
        if (arr[medio] < chiave) inizio = medio + 1;
        else fine = medio - 1;
    }
    return -1;
}
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Nascita**: ideato nel 1979 da **Bjarne Stroustrup** come *"C with Classes"* per unire velocità del C e OOP.
> - **Esecuzione Nativa**: compilato **direttamente in codice macchina** (nessuna macchina virtuale / VM).
> - **Zero-Overhead**: non paghi in memoria o tempo di calcolo ciò che non utilizzi.
> - **Memoria Deterministica (RAII)**: nessun Garbage Collector; liberazione automatica nel distruttore.
> - **Pipeline di Compilazione**: **Preprocessore (`#`)** → **Compilatore (`.obj`)** → **Linker (`.exe`)**.

Il **C++** è uno dei linguaggi di programmazione più potenti e diffusi nei settori in cui sono richieste **massime prestazioni**, **bassa latenza** e **controllo diretto sull'hardware** (game engine come Unreal Engine, sistemi operativi, software aerospaziale, finanza ad alta frequenza e intelligenza artificiale).

---

1. Caratteristiche Fondamentali

A. Linguaggio Compilato Nativo

A differenza di linguaggi interpretati (come Python o JavaScript) o basati su bytecode e macchine virtuali (come Java o C#), il codice C++ viene compilato **direttamente in linguaggio macchina binario** per la CPU target. Non esiste alcun livello intermedio di emulazione o virtual machine.

B. Tipizzazione Statica e Forte

- **Statica**: il tipo di ogni variabile deve essere noto prima della compilazione e non può mutare a runtime.
- **Forte**: il compilatore impedisce conversioni implicite pericolose tra tipi non correlati.

C. Linguaggio Multi-Paradigma

Il C++ permette di combinare liberamente più stili di programmazione:

1. **Procedurale / Imperativo**: codice strutturato in funzioni e flussi sequenziali (stile C).
2. **Orientato agli Oggetti (OOP)**: classi, incapsulamento, ereditarietà e polimorfismo.
3. **Generico (Template)**: algoritmi e strutture dati indipendenti dal tipo di dato con la STL (*Standard Template Library*).
4. **Funzionale**: supporto moderno a funzioni lambda ed espressioni chiuse.

> 🎯 **Il Principio del "Zero-Overhead"**
>
> *"Ciò che non usi, non lo paghi. E ciò che usi, non potresti scriverlo a mano in modo più efficiente."*
> Se una funzionalità avanzata del linguaggio non viene impiegata, il compilatore non genera un solo byte di overhead in memoria o in tempo di calcolo.

---

2. Il Ciclo di Compilazione (Dal Sorgente all'Eseguibile)

> ❓ **Domanda d'Esame: Quali sono le tre fasi di compilazione di un programma C++?**

```
flowchart LR
    SRC["File Sorgente (.cpp)<br>+ Header (.h)"]
    PRE["1. Preprocessore<br>(#include, #define)"]
    CMP["2. Compilatore<br>(Analisi sintattica e traduzione)"]
    OBJ["File Oggetto (.obj / .o)<br>(Codice macchina binario)"]
    LNK["3. Linker (Collegatore)<br>(Risoluzione simboli esterni)"]
    LIB["Librerie Standard<br>(cout, sqrt, ...)"]
    EXE["Programma Finale<br>(.exe / ELF)"]

    SRC --> PRE
    PRE --> CMP
    CMP --> OBJ
    OBJ --> LNK
    LIB --> LNK
    LNK --> EXE
```

1. Preprocessing (Preprocessore)

- Interpreta tutte le direttive che iniziano con `#`.
- Sostituisce `#include <file>` con il testo effettivo del file header.
- Espande le costanti e macro dichiarate con `#define`.
- Elimina tutti i commenti (`//` e `/* */`).
- Il risultato è un'unica sequenza continua di codice C++ detta **Unità di Traduzione**.

2. Compilazione (Compilatore)

- Effettua l'analisi lessicale, sintattica e semantica.
- Ottimizza il codice per la CPU di destinazione.
- Traduce il codice C++ in istruzioni in linguaggio macchina salvate nei **File Oggetto** (`.obj` su Windows, `.o` su Linux).

3. Linking (Collegatore / Linker)

- Unisce tra loro i vari file oggetto che compongono il progetto.
- Risolve i riferimenti esterni (es. individua dove risiede la funzione `std::cout` nella libreria standard compilata).
- Genera il file binario finale eseguibile (`.exe`).

> 🖼️ **Placeholder Immagine: Schema visuale della pipeline di compilazione del C++**
>
> *Suggerimento per Obsidian: inserisci qui un diagramma riassuntivo del processo gcc / clang con i vari file intermedi (.i, .s, .o, .exe).*
> `![[Pasted image pipeline_compilazione.png|600]]`

---

3. L'Evoluzione degli Standard C++

Standard	Anno	Novità Principali
C++98 / C++03	1998/2003	Prima standardizzazione formale ISO, introduzione della STL e template.
C++11	2011	**Rivoluzione del linguaggio**: `auto`, `nullptr`, *range-based for*, lambda, smart pointer.
C++14 / C++17	2014/2017	`std::optional`, `std::filesystem`, `if constexpr`, miglioramenti alle lambda.
C++20	2020	*Concepts*, *Ranges*, *Coroutines*, Moduli (`import`).

---

4. Confronto tra Linguaggi

Aspetto	C	C++	Java / Python / C#
Paradigmi	Solo procedurale	Multi-paradigma (OOP, Generico, Funzionale)	Prevalentemente OOP / Multi-paradigma
Classi ed Oggetti	No	Sì (pieno supporto)	Sì (obbligatorio in Java)
Gestione Memoria	Manuale (`malloc` / `free`)	**Deterministica (RAII)**	Automatica con Garbage Collector
Velocità Esecuzione	Massima	**Massima (pari al C)**	Minore (dovuta a VM / Garbage Collector)
Esecuzione	Binario nativo	Binario nativo	Bytecode interpretato o JIT compilato